

BÁO CÁO THỰC HÀNH

LAB04 - CROSS-SITE REQUEST FORGERY

Môn: Mạng máy tính

Sinh viên: Lê Minh

MSSV: 21127645

Môi trường: localhost - dữ liệu giả lập

1. Mục tiêu và môi trường thực hành

Mục tiêu là giải thích vì sao browser tự gửi session cookie, tái hiện lỗi email thiếu CSRF token và kiểm chứng synchronizer token kết hợp exact Origin/Referer validation.

Victim Application chạy tại <http://127.0.0.1:5004>; Demo Page cố định tại <http://127.0.0.1:9004>. Tài khoản victim dùng dữ liệu SQLite giả lập. Không có website, email hay tài khoản thật.

2. Kịch bản và các bước thực hiện

2.1 Request hợp lệ ban đầu

Đăng nhập victim và gửi request đổi email cùng origin để xác nhận session và trạng thái ban đầu.

ẢNH 01/07

Tên file: 01_login_valid_request.png

Tiêu đề ảnh: Chứng minh victim đã đăng nhập và request đổi email hợp lệ ban đầu.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5004/login> rồi <http://127.0.0.1:5004/vulnerable/change-email>

Thao tác: victim / Victim123!; email=victim_initial@lab.local | Đăng nhập; Đổi email

Panel hoặc DevTools tab: DevTools Network > request POST; ghép với Dashboard.

Nội dung bắt buộc phải thấy: Session victim, POST /vulnerable/change-email và email mới trên Dashboard.

Kết quả mong đợi: Request cùng origin hợp lệ ban đầu đổi email thành công.

Caption: Victim đăng nhập và gửi request đổi email hợp lệ ban đầu.

Hình 1. Victim đăng nhập và gửi request đổi email hợp lệ ban đầu.

2.2 Form giả lập và vulnerable CSRF

Demo Page chứa form POST local cố định đến /vulnerable/change-email, không biết password hoặc token của victim.

ẢNH 02/07

Tên file: 02_csrf_demo_form.png

Tiêu đề ảnh: Chứng minh trang Demo Page có form CSRF local cố định.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:9004/attack/vulnerable-email>

Thao tác: Form cố định gửi email=demo_changed@lab.local đến 127.0.0.1:5004. | Chưa gửi; chỉ mở trang

Panel hoặc DevTools tab: Form Inspector hoặc DevTools Elements.

Nội dung bắt buộc phải thấy: method POST, action /vulnerable/change-email, hidden/input email cố định và cảnh báo chỉ localhost.

Kết quả mong đợi: Mã/giao diện form local đúng kịch bản, không có target tùy ý.

Caption: Trang Demo Page chứa form CSRF local cố định.

Hình 2. Trang Demo Page chứa form CSRF local cố định.

Khi victim còn đăng nhập và chủ động bấm gửi form demo, route vulnerable chỉ dựa vào session cookie nên email bị đổi.

ẢNH 03/07

Tên file: 03_csrf_vulnerable_changed.png

Tiêu đề ảnh: Chứng minh CSRF vulnerable đổi email của victim.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:9004/attack/vulnerable-email`

Thao tác: `email=demo_changed@lab.local.` | Gửi form rồi xác nhận

Panel hoặc DevTools tab: DevTools Network; ghép response/trace với Dashboard Victim.

Nội dung bắt buộc phải thấy: POST được gửi, cookie/session hiện diện trong trace và email đổi thành `demo_changed@lab.local.`

Kết quả mong đợi: Route vulnerable chấp nhận request không có token và state thay đổi.

Caption: CSRF vulnerable dùng session cookie để đổi email victim.

Hình 3. CSRF vulnerable dùng session cookie để đổi email victim.

2.3 Kiểm chứng bản secure

Request thiếu hoặc sai token trả 403 và database không đổi.

ẢNH 04/07

Tên file: 04_csrf_secure_403.png

Tiêu đề ảnh: Chứng minh token thiếu hoặc sai bị từ chối và state không đổi.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:9004/attack/secure-email` hoặc `/attack/bad-token`

Thao tác: Thiếu token hoặc token sai; `email=blocked@lab.local.` | Gửi form rồi xác nhận

Panel hoặc DevTools tab: Network Response và CSRF/State Inspector.

Nội dung bắt buộc phải thấy: HTTP 403, token missing/invalid, database update skipped và email không đổi.

Kết quả mong đợi: Bản secure từ chối request trước mutation.

Caption: CSRF token thiếu hoặc sai bị từ chối, state không đổi.

Hình 4. CSRF token thiếu hoặc sai bị từ chối, state không đổi.

Request từ Demo Page còn bị exact Origin/Referer validation từ chối trước mutation.

ẢNH 05/07

Tên file: 05_origin_blocked.png

Tiêu đề ảnh: Chứng minh Origin/Referer validation chặn request từ Demo Page.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:9004/attack/secure-email`

Thao tác: `Origin=http://127.0.0.1:9004;` token không phải điều kiện quyết định ảnh này. | Gửi form rồi xác nhận

Panel hoặc DevTools tab: Origin Inspector và Network Response.

Nội dung bắt buộc phải thấy: submitted origin 9004, expected origin 5004, exact match=false, HTTP 403 và state không đổi.

Kết quả mong đợi: Request khác origin bị chặn trước khi cập nhật database.

Caption: Exact Origin/Referer validation chặn request từ Demo Page.

Hình 5. Exact Origin/Referer validation chặn request từ Demo Page.

Request cùng origin có token hợp lệ cập nhật email; token được rotate sau thành công.

ẢNH 06/07

Tên file: 06_csrf_secure_success.png

Tiêu đề ảnh: Chứng minh token hợp lệ cho phép request và được rotate.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5004/secure/change-email`

Thao tác: `email=secure_success@lab.local`; dùng hidden token do server cấp. | Đổi email an toàn

Panel hoặc DevTools tab: CSRF Token, Origin và State Inspector.

Nội dung bắt buộc phải thấy: Origin hợp lệ, token valid, email đổi thành công và `rotation status=rotated`.

Kết quả mong đợi: Request có token/session/origin hợp lệ thành công; token cũ không còn dùng lại.

Caption: Bản secure chấp nhận token hợp lệ và rotate token sau mutation.

Hình 6. Bản secure chấp nhận token hợp lệ và rotate token sau mutation.

3. Nguyên nhân kỹ thuật

Browser quản lý cookie và tự gắn cookie khi policy cho phép. Route vulnerable coi cookie xác thực là đủ bằng chứng về ý định, không yêu cầu token hoặc kiểm tra nguồn request. SOP thường ngăn trang khác đọc response nhưng không ngăn form HTML gửi request.

```
# Vulnerable: chỉ dựa vào session
@login_required
def vulnerable_change_email():
    update_email(session['user_id'], request.form['email'])
```

4. Kết quả và bằng chứng

Luồng vulnerable đổi state mà không có token. Luồng secure từ chối token thiếu/sai và Origin khác; chỉ request có session, Origin/Referer hợp lệ và token đúng mới cập nhật. Audit/trace ghi decision nhưng không lưu token/cookie đầy đủ.

ẢNH 07/07

Tên file: 07_audit_test_report.png

Tiêu đề ảnh: Chứng minh audit, kiểm thử và report artifacts.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5004/audit-logs` và terminal tại Lab04

Thao tác: `python -m pytest -q`; `python scripts/generate_report.py` | Filter nếu cần; chạy hai lệnh terminal

Panel hoặc DevTools tab: Audit Inspector; terminal phải đọc được output thật.

Nội dung bắt buộc phải thấy: Audit có `vulnerable_email_changed`, `csrf_token_invalid/origin denied`, `secure_email_changed`; terminal có `pytest summary` và tên DOCX/PDF.

Kết quả mong đợi: Audit, pytest và report artifacts xuất hiện trong một ảnh đọc được.

Caption: Audit log cùng kết quả kiểm thử và report artifacts của Lab04.

Hình 7. Audit log cùng kết quả kiểm thử và report artifacts của Lab04.

5. Mức độ ảnh hưởng

CSRF có thể làm sai tính toàn vẹn dữ liệu bằng quyền của victim. Đổi email trong hệ thống thật có thể hỗ trợ chiếm quyền; logout gây gián đoạn; thao tác tài chính có thể nghiêm trọng. Lab chỉ đổi dữ liệu giả lập local.

6. Bản vá và cách phòng chống

Dùng token ngẫu nhiên duy nhất theo session, kiểm tra ở server trước mutation và rotate sau thành công. Kèm `SameSite=Lax/Strict` phù hợp, `exact Origin/Referer`, `POST-only` và `re-authentication` cho thao tác nhạy cảm. CAPTCHA và CORS không phải biện pháp chính.

```
# Secure: kiểm tra trước UPDATE
validate_origin_or_referer(request)
validate_csrf_token(session['csrf_token'], request.form['csrf_token'])
update_email(session['user_id'], validated_email)
```

```
rotate_csrf_token(session)
```

7. Trả lời các câu hỏi báo cáo trong BaiTapTopic04.docx

1. Browser tự gửi cookie vì cookie jar và matching policy do browser quản lý; ứng dụng không cần tự thêm cookie vào từng form.
2. CSRF không cần biết mật khẩu vì victim đã có session được xác thực; request lợi dụng credential đó.
3. Trang tạo request cross-origin thường không đọc được response do SOP, nhưng request vẫn có thể thay đổi state.
4. CSRF ép browser thực hiện hành động bằng credential sẵn có; XSS chạy script trong trusted origin và có thể đọc token trong DOM.
5. Không dùng GET cho state change vì link, prefetch, cache hoặc crawler có thể kích hoạt ngoài ý muốn; thao tác phải dùng POST và kiểm tra token.

8. Kết quả kiểm thử

Pytest summary đọc từ evidence/logs/pytest.txt:

```
..... [ 82%] ..... [100%]  
87 passed in 31.81s
```

Coverage summary đọc từ evidence/logs/coverage.txt:

```
..... [ 82%] ..... [100%]  
===== tests coverage =====  
coverage: platform win32, python 3.12.6-final-0 _____ Name Stmts Miss Cover Missing  
----- audit_service.py 19 0 100% auth.py 36 0 100%  
csrf_service.py 32 1 97% 40 database.py 41 3 93% 56-58 origin_service.py 49 4 92% 56-57, 72-73  
trace_service.py 35 1 97% 90 ----- TOTAL 212 9 96% 87  
passed in 32.80s
```

9. Kết luận

CSRF tồn tại khi server nhầm credential tự động gửi với ý định của người dùng. Token server-side là lớp chính; Origin/Referer, SameSite, POST-only, re-authentication và audit là các lớp bổ sung.